

JVM-прапорці та environment variables у проєкті: простий конспект

Пояснення ризиків, легітимних сценаріїв і політики jvm-attestor

Матеріал базується на фактичному списку перевірок у репозиторії.

1. Межі цього конспекту

Цей документ описує весь список, який реально перевіряє `spire-jvm-attestor` у файлі [internal/checkers/antitamper.go](https://github.com/elastic/spire/blob/main/internal/checkers/antitamper.go). Це не список «усіх небезпечних параметрів Java у світі». Це саме security-політика даного проєкту.

Плагін не намагається довести, що конкретний інструмент «поганий». Він застосовує просте правило zero trust: якщо JVM має параметр, який може завантажити агент, увімкнути налагодження, дати керування процесом або приховано додати опції, workload не отримує SVID.

JVM command line / environment / Attach socket
↓
jvm-attestor читає /proc/<pid>
↓
чисто → jvm:agent_flags_clean=true → SVID можливий
знайдено ризик → ...=false / помилка → SVID відхилено

Важливо: «легітимний» не означає «дозволений цією політикою». Профайлер, APM-агент або JMX можуть бути корисними та законними інструментами, але водночас мають високий рівень доступу до JVM. У production-профілі цього проєкту їх наявність є підставою відмовити в ідентичності.

2. Як саме працює перевірка

1. Плагін читає `/proc/<pid>/cmdline` і розділяє аргументи за нульовим байтом. 2. Він шукає не точне значення, а префікс: наприклад, будь-який аргумент, що починається з `-javaagent:`. 3. Потім він читає `/proc/<pid>/environ`. Будь-яка з чотирьох перелічених змінних із непорожнім значенням є сигналом ризику — незалежно від її конкретного вмісту. 4. Також перевіряється `/proc/<pid>/root/tmp/.java_pid*`: це ознака JVM Attach API socket.

У стандартній конфігурації проєкту `block_on_attach_socket = true`. Тому attach socket викликає помилку attestation, а не просто selector. Для прапорців і env змінних результат — `jvm:agent_flags_clean=false` плюс пояснювальний selector (`jvm:suspicious_flag=...` або `jvm:suspicious_env=...`).

3. Огляд усіх перевірок

Категорія	Що саме блокується	Чому це чутливо
Java agent	<code>-javaagent:</code>	код у JAR агента запускається до застосунку й може інструментувати класи
Native agent	<code>-agentlib:</code> , <code>-agentpath:</code>	завантажується нативна бібліотека <code>.so</code> , що має дуже широкий доступ у процесі
Remote debugging	<code>-Xrunjdw:</code> , <code>-Xdebug</code>	JDWP дозволяє налагодження JVM; відкритий порт debug особливо ризикований
Dynamic attach	<code>-Djdk.attach.allowAttachSelf=true</code> та <code>.java_pid*</code>	дає змогу під'єднувати інструменти до живої JVM і завантажувати агент у runtime
JMX management	<code>-Dcom.sun.management.jmxremote...</code>	відкриває/налаштовує інтерфейс керування JVM
Hidden option injection	<code>JAVA_TOOL_OPTIONS</code> , <code>_JAVA_OPTIONS</code> , <code>JDK_JAVA_OPTIONS</code> , <code>IBM_JAVA_OPTIONS</code>	опції можуть бути додані без зміни явної команди <code>java ...</code>

4. -javaagent:<jar>

Що це. Java agent — JAR, який JVM завантажує на старті. Він має **Premain-Class**; JVM викликає його метод **premain** до **main** застосунку. Java Instrumentation API дозволяє агенту спостерігати за класами та перетворювати їхній bytecode.

Чим може бути небезпечний. Агент виконується всередині того самого процесу з правами застосунку. Він потенційно бачить дані в пам'яті, може змінювати поведінку класів, перехоплювати виклики або підмінити логіку. Це сильний механізм, тому сам факт непідтвердженого агента є порушенням цілісності workload'a.

Коли легітимний. APM/observability (наприклад, трасування), code coverage у CI, профайлери, security agents, інструменти тестування та іноді Java-framework agents. Такі сценарії реальні, але для захищеного production workload їх треба окремо погоджувати, фіксувати версію/хеш агента і тестувати в окремому профілі.

Політика проекту. Блокується будь-який префікс **-javaagent:**. Навіть «добрий» агент у базовому custom-jvm профілі дасть **jvm:agent_flags_clean=false** і не отримає SVID.

5. -agentlib: і -agentpath:

Ці два параметри завантажують native JVM TI (JVMTI) agent. На Linux це зазвичай shared library **.so**.

Прапорець	Відмінність
-agentlib:<name>	JVM шукає бібліотеку за її ім'ям у системних шляхах завантаження
-agentpath:/повний/шлях/lib.so	JVM завантажує бібліотеку за явно заданим повним шляхом

Чому ризик вищий. Native-код не обмежений моделлю безпеки Java й працює в адресному просторі JVM. JVM TI створена саме для debugging, profiling, monitoring, coverage та інструментування, тобто для дуже глибокого спостереження і контролю процесу.

Коли легітимні. JDWP, профайлери, native monitoring agents, деякі діагностичні й performance tools. Наприклад, тест у репозиторії створює безпечний тестовий **.so**, щоб JVM запустилась і attester дійсно побачив параметр. Це не означає, що такий агент допускається в production.

Політика проекту. Блокується весь клас **-agentlib:** і **-agentpath:**, без allow-list конкретних бібліотек.

6. JDWP: -Xrunjdpw: та -Xdebug

JDWP (Java Debug Wire Protocol) — протокол, яким IDE або debugger спілкується з JVM. Типова форма налаштовує сокет (**dt_socket**), режим server/client, адресу/порт та поведінку запуску.

-Xrunjdpw: — історичний спосіб запуску JDWP agent. **-Xdebug** — історичний/сумісний debug-прапорець, який часто зустрічається поряд із JDWP. У сучасних JDK конкретна поведінка залежить від версії, але для attestation важливий сам сигнал: JVM стартувала з debug-параметрами.

Ризик. Debugger може читати стан програми, зупиняти потоки, ставити breakpoints і змінювати хід виконання. Якщо debug endpoint доступний сторонній особі, це може стати повним компромісом застосунку. Навіть localhost-port у контейнері не слід автоматично вважати безпечним: мережеві namespace, port-forward або доступ з вузла змінюють межі доступу.

Легітимне використання. Локальна розробка, integration testing, коротка контрольована діагностика інциденту. У цьому випадку краще використовувати окремий dev/debug deployment без production SVID, із закритим портом і коротким TTL.

7. -Djdk.attach.allowAttachSelf=true і Attach API socket

Attach API дозволяє Java-інструменту приєднатись до вже запущеної JVM. Це може застосовуватись для діагностики, JConsole/JMC, профайлінгу або динамічного завантаження агента. Під час attach JVM створює Unix-domain socket на кшталт `/tmp/.java_pid<PID>`.

`-Djdk.attach.allowAttachSelf=true` знімає обмеження на attach процесу до самого себе в ситуаціях, де це заборонено за замовчуванням. Це не є «вірусним» параметром, але зменшує захисну межу навколо живого процесу.

Чому плагін перевіряє і параметр, і socket. Параметр показує намір/можливість self-attach, а socket показує, що механізм attach доступний зараз. Перевірка socket корисна також тоді, коли attach ініціював зовнішній інструмент, а не командний рядок застосунку.

Легітимне використання. Runtime profiling, JFR/діагностика, JConsole, Byte Buddy та деякі бібліотеки, що встановлюють agent динамічно. У цьому проєкті socket за замовчуванням жорстко блокує SVID. Якщо потрібна діагностика, краще виконувати її на окремій debug-копії workload'a, а не послаблювати базове правило непомітно.

8. -Dcom.sun.management.jmxremote...

JMX (Java Management Extensions) — API керування й моніторингу JVM: memory, threads, MBeans, інколи операції застосунку. Префікс `-Dcom.sun.management.jmxremote` увімкнює або налаштовує management agent; подальші властивості можуть задавати порт, TLS, автентифікацію тощо.

Ризик. Remote JMX створює адміністративний інтерфейс. Небезпечні конфігурації — відкритий порт, слабка/відсутня автентифікація чи вимкнене TLS. Через широкий префікс плагін не аналізує, чи конкретна JMX-конфігурація добра: він просто ставиться до неї як до відхилення від baseline.

Легітимне використання. Локальна діагностика або контрольований production monitoring із TLS, клієнтськими сертифікатами, автентифікацією і network policy. Для цього проєкту бажані альтернативи — application metrics до Prometheus і окремий моніторинговий контур, а не неочікуваний JMX у workload.

9. Environment variables, що можуть додавати JVM options

Змінні середовища небезпечні не самою назвою. Їхня небезпека в тому, що вони можуть додати JVM-опції, хоча Dockerfile або Kubernetes `command` виглядають чистими. Саме тому плагін блокує будь-яке непорожнє значення цих ключів.

Змінна	Хто зазвичай її читає	Ризик і легітимне застосування
<code>JAVA_TOOL_OPTIONS</code>	JVM/launch mechanism, зокрема в інструментованих середовищах	зручна глобальна ін'єкція агентів, системних властивостей і debug flags; легітимна для CI, профайлінгу, централізованого налаштування
<code>_JAVA_OPTIONS</code>	де-факто HotSpot/OpenJDK-середовища	може непомітно додати options до JVM; легітимна для тимчасових dev/desktop налаштувань, але погано прозора в production

Змінна	Хто зазвичай її читає	Ризик і легітимне застосування
<code>JDK_JAVA_OPTIONS</code>	launcher <code>java</code> у JDK	значення додаються перед явними аргументами <code>java</code> ; JDK обмежує частину небезпечних launcher-опцій, але не перетворює змінну на безпечну
<code>IBM_JAVA_OPTIONS</code>	IBM Semeru / OpenJ9-сумісні середовища	vendor-specific глобальні JVM options; потрібна для portability, але теж приховує фактичну конфігурацію

Чому тут бувають false positives

Наприклад, `JAVA_TOOL_OPTIONS=-Duser.timezone=UTC` може бути цілком безпечним, але цей плагін усе одно відмовить у SVID, бо не інтерпретує значення. Це свідомий компроміс: просте й перевірюване правило замість складного allow-list, який легко пропустити або обійти.

Якщо команда справді потребує легітимного параметра, варіанти рішення треба погоджувати як зміну політики: окремий dev/observability profile, конкретний approved image, контрольований deployment або розширення плагіна з вузьким allow-list. Не варто просто переносити опції в env variable: плагін перевіряє і `cmdline`, і `environ`.

10. Як це пов'язано зі SPIRE і SVID

Плагін не завершує Java-процес і не «лікує» його. Він лише повертає facts — selectors. Потім SPIRE порівнює selectors із registration entry.

```
java запускається
→ SPIRE Agent просить jvm-attestor перевірити PID
→ знайдено -javaagent:...
→ jvm:agent_flags_clean=false
→ entry вимагає jvm:agent_flags_clean=true
→ selectors не збігаються
→ SVID не видається
→ mTLS до іншого сервісу не може пройти після нового handshake
```

Це також пояснює дизайн attack-тестів: вони запускають compromised pod до першого запиту SVID. Старий SVID і вже встановлене mTLS-з'єднання можуть ще певний час існувати, але новий або перезапущений workload не отримає ідентичності.

11. Практичні правила для команди

1. Production baseline: не додавайте перелічені flags чи env variables у `Deployment`, `Dockerfile` або CI injectors без окремого рішення. 2. Розробка: debug/JMX/profiler запускайте на dev-копії сервісу, яка не претендує на production SPIRE identity. 3. Спостережуваність: за можливості використовуйте Prometheus metrics і application logs. Якщо потрібен Java agent/JMX, документуйте його версію, джерело, ціль, мережеві обмеження та lifecycle. 4. Діагностика: перевіряйте і явну команду Java, і environment. Не довіряйте лише Dockerfile. 5. Зміна політики: не вимикайте attestor «на хвилинку». Спершу визначте, який саме легітимний інструмент потрібен, який мінімальний доступ він потребує та як звузити дозволи.

12. Довідка та джерела

- Реальна політика проєкту: [spire-jvm-attestor/internal/checkers/antitamper.go](https://github.com/spiffe/spire-jvm-attestor/blob/main/internal/checkers/antitamper.go).
- Java Instrumentation (`-javaagent`): <https://docs.oracle.com/en/java/javase/21/docs/api/java.instrument/java/lang/instrument/package-summary.html>
- JVM TI (`-agentlib`, `-agentpath`): <https://docs.oracle.com/en/java/javase/20/docs/specs/jvmti.html>

- Java launcher і **JDK_JAVA_OPTIONS**:
<https://docs.oracle.com/en/java/javase/17/docs/specs/man/java.html>
- JMX management: <https://docs.oracle.com/en/java/javase/24/management/monitoring-and-management-using-jmx-technology.html>
- Attach/JConsole: <https://docs.oracle.com/en/java/javase/26/management/using-jconsole.html>

Короткий словник

Термін	Значення
JVM option / flag	параметр, із яким запускають JVM
Java agent	JAR, що виконується разом із JVM та може інструментувати код
Native agent	.so бібліотека, завантажена в процес JVM через JVM TI
JDWP	протокол віддаленого налагодження Java
JMX	API керування й моніторингу Java
Attach API	спосіб під'єднатися інструментом до живої JVM
Unix socket	локальний файл-канал для обміну даними між процесами
SVID	короткоживуча цифрова ідентичність SPIFFE/SPIRE
Selector	факт про workload, за яким SPIRE приймає рішення

Головна думка: усі ці інструменти можуть бути корисними. Але вони дають глибокий доступ до JVM або приховано змінюють спосіб її запуску. Тому ця робота трактує їх як події, що потребують окремої довіри, а не як нормальний стан production workload'a.